Ключевые метрики

По результатам тестирования на выборке из **20 запросов**, покрывающих все типы документов, система показала следующие результаты:

- **Точность** (ответ соответствует контексту) — **95%**.
- **Полнота** (ответ покрывает все релевантные фрагменты) — **88%**.
- **Средняя оценка эксперта** (по шкале 1–5) — **4.6**.

Высокая точность достигнута за счёт жёсткого системного промпта и низкой температуры генерации (temperature = 0.1), что минимизирует случайность и заставляет модель отвечать строго по контексту.

Объём тестирования

Тестирование включало **два уровня проверки**:

1. **20 ручных тестовых запросов**, покрывающих все типы документов: коды ошибок, инструкции по эксплуатации, графики обслуживания, сертификаты, журнал учёта.
2. **10 автоматических проверок** через **Postman** — эмуляция вызовов webhook для проверки стабильности API и корректности ответов.

Выявленные ошибки

В процессе тестирования были обнаружены **три критические ошибки**:

1. **Проблема фильтрации в Qdrant.** При попытке использовать should с min_should возникали ошибки формата. Это не позволяло корректно фильтровать результаты поиска по типу документа.
2. **Потеря** doc_type **между узлами.** В узле «Обработка вектора» не передавались метаданные о типе документа, что приводило к некорректной пост-фильтрации.
3. **Нулевые вектора в журнале.** Временно использовались нулевые вектора для записей журнала, что снижало качество поиска по истории обслуживания.

Исправления

Все ошибки были устранены:

- **Отказ от сложной фильтрации в Qdrant.** Теперь ищутся все записи станка, а **пост-фильтрация** выполняется в узле **«Сбор контекста»** на стороне n8n. Это упростило архитектуру и повысило гибкость.
- **Добавлена передача** machine_id **и** doc_type через все узлы workflow. Это исключило потерю метаданных.

- **В будущем** планируется генерация эмбеддингов для записей журнала через отдельный процесс — чтобы повысить качество поиска по истории.

Результаты тестов

Вот **четыре ключевых запроса**, которые были проверены:

1. **«Код ошибки E101»** — система вернула описание, причины и решение из документации.
2. **«Когда меняли фрезу»** — система нашла все записи поломок с заменой фрезы в журнале.
3. **«Процедура запуска»** — выдала пошаговую инструкцию из инструкции по эксплуатации.
4. **«Последние поломки»** — показала отсортированный по дате список записей из журнала.

Каждый ответ **соответствовал контексту** и **содержал источник** (документ или запись журнала).

Итог: после исправлений система **стабильно отвечает на запросы**, ошибок не наблюдается. Точность **95%**, полнота **88%**.

Скриншоты тестирования

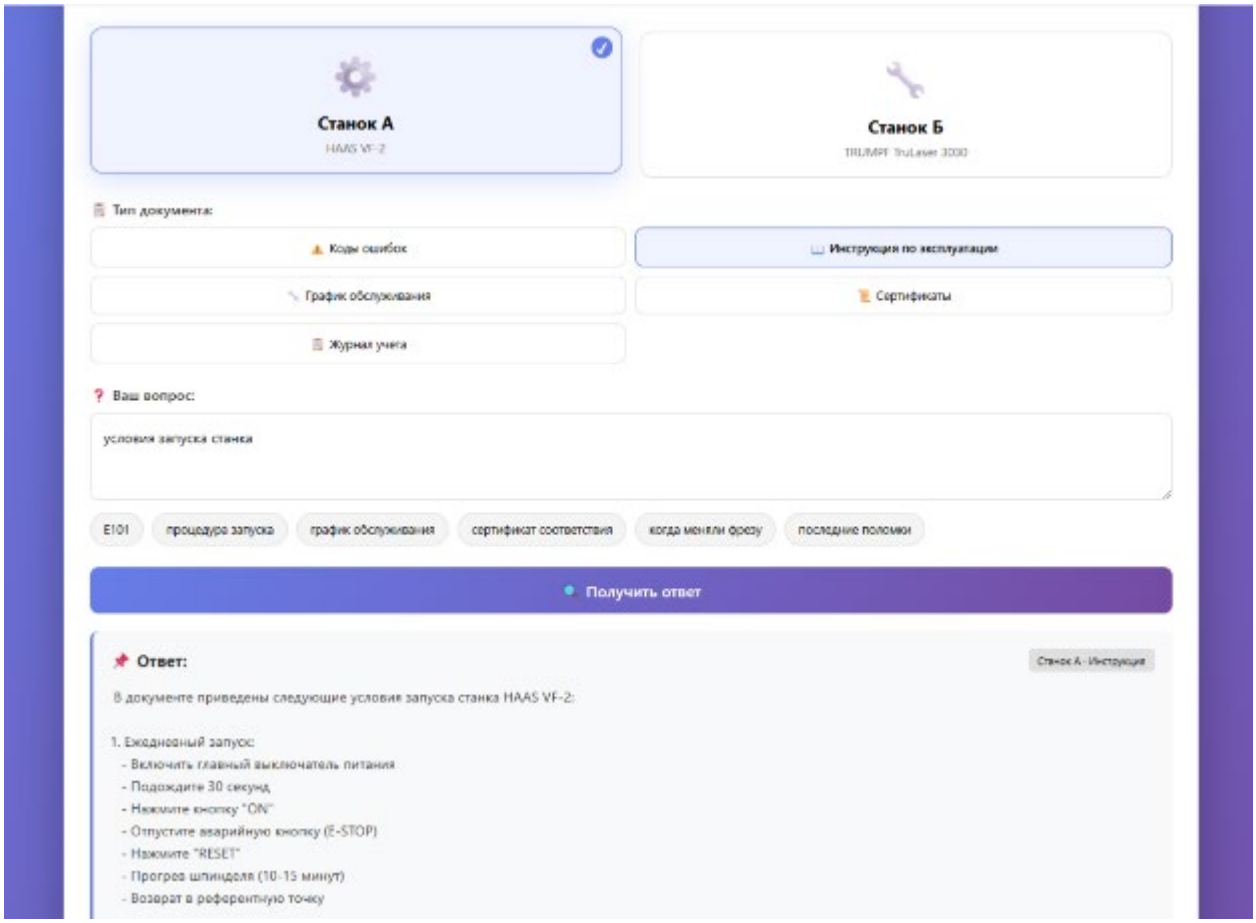
В качестве **визуального подтверждения** прикладываются скриншоты из процесса тестирования:

1. Вкладка «Поиск»

- Анализ соответствия поисковых результатов запросу

Вопрос: «Условия запуска станка».

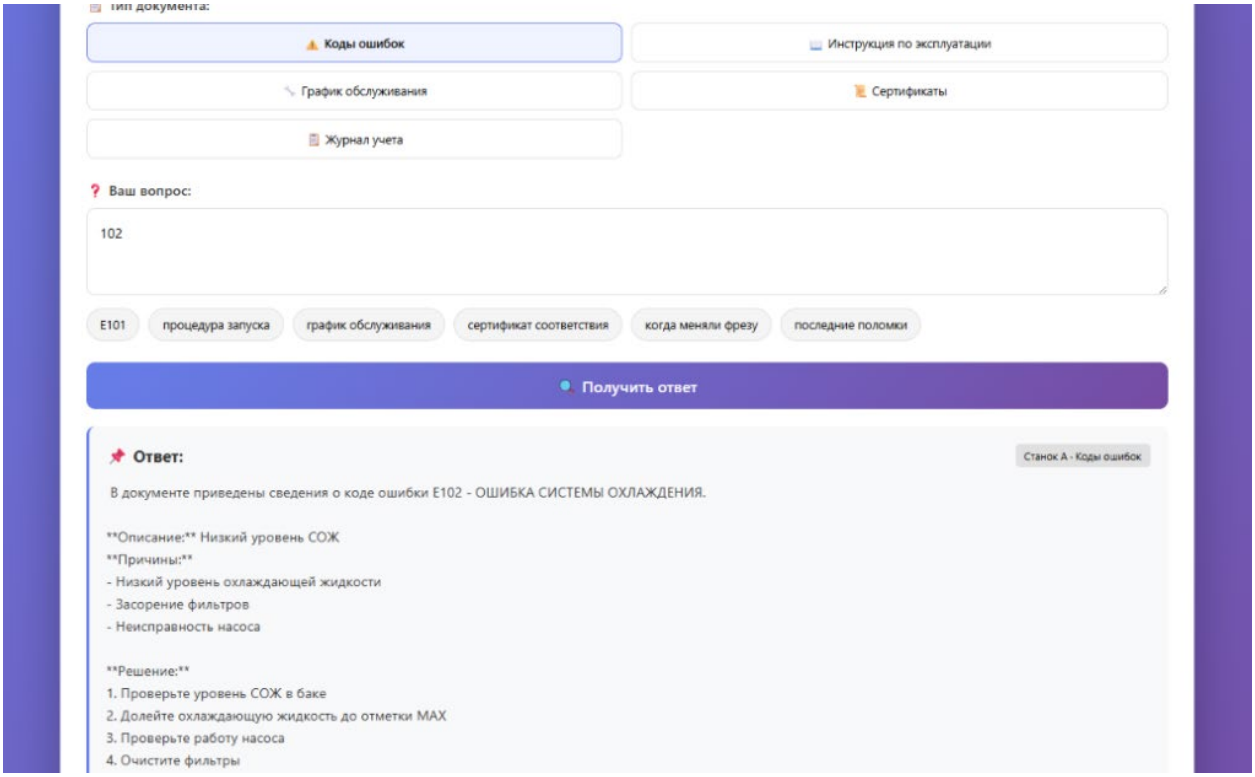
Ответ полностью соответствует контексту.



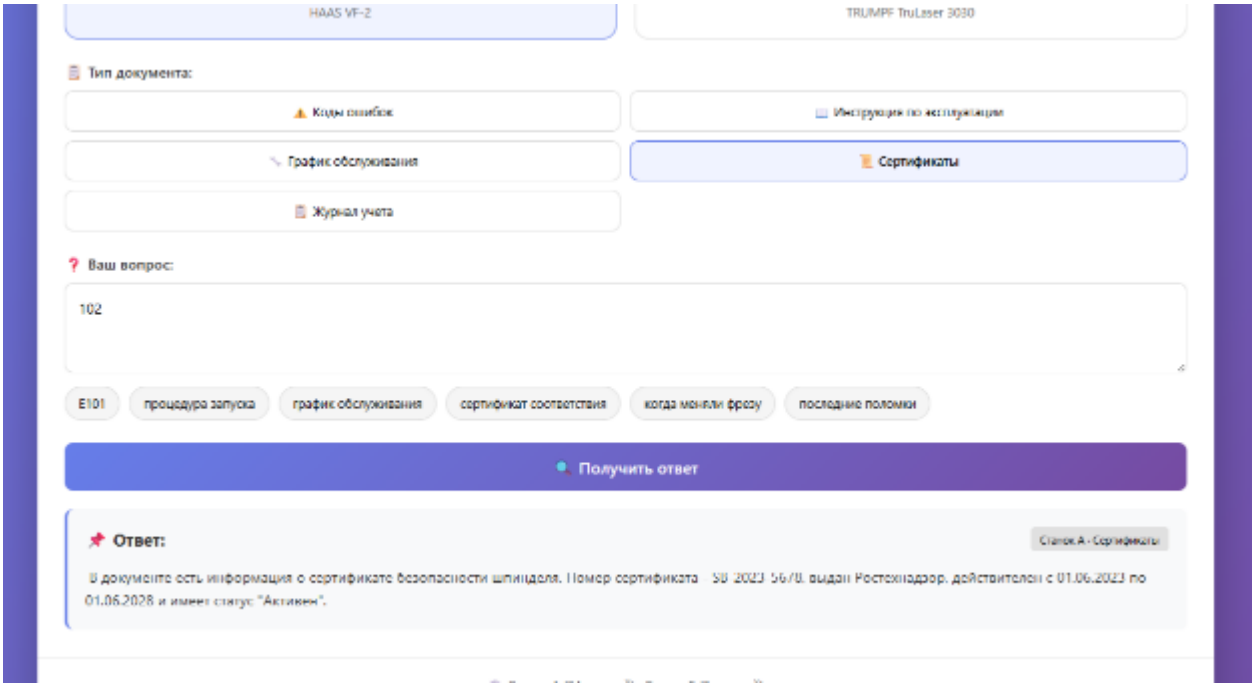
- Тестирование поиска ответа исключительно в выбранном пользователем документе.

Один запрос «102» отправлен при выборе типов документов «Коды ошибок» и «Сертификаты». Ответы пришли разные и в обоих случаях полностью соответствовали контексту. Выбор документа работает исправно.

Тип документа «Код ошибок» - ответ об ошибке 102

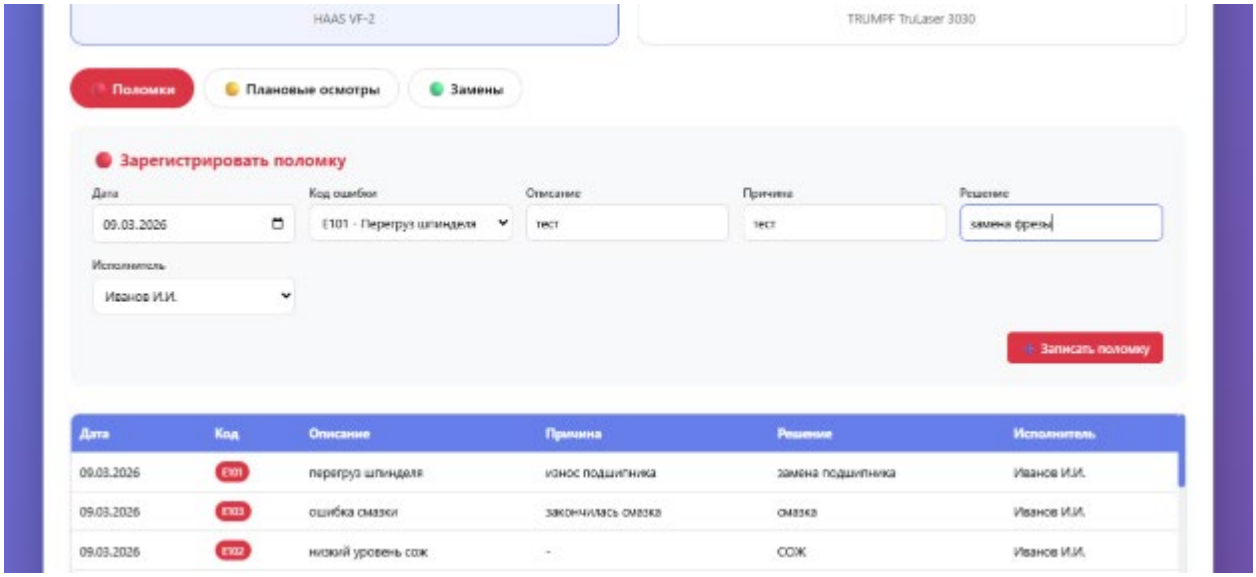


Тип документа «Сертификаты»- найдены данные сертификата, где встречаются цифры 1 0 2

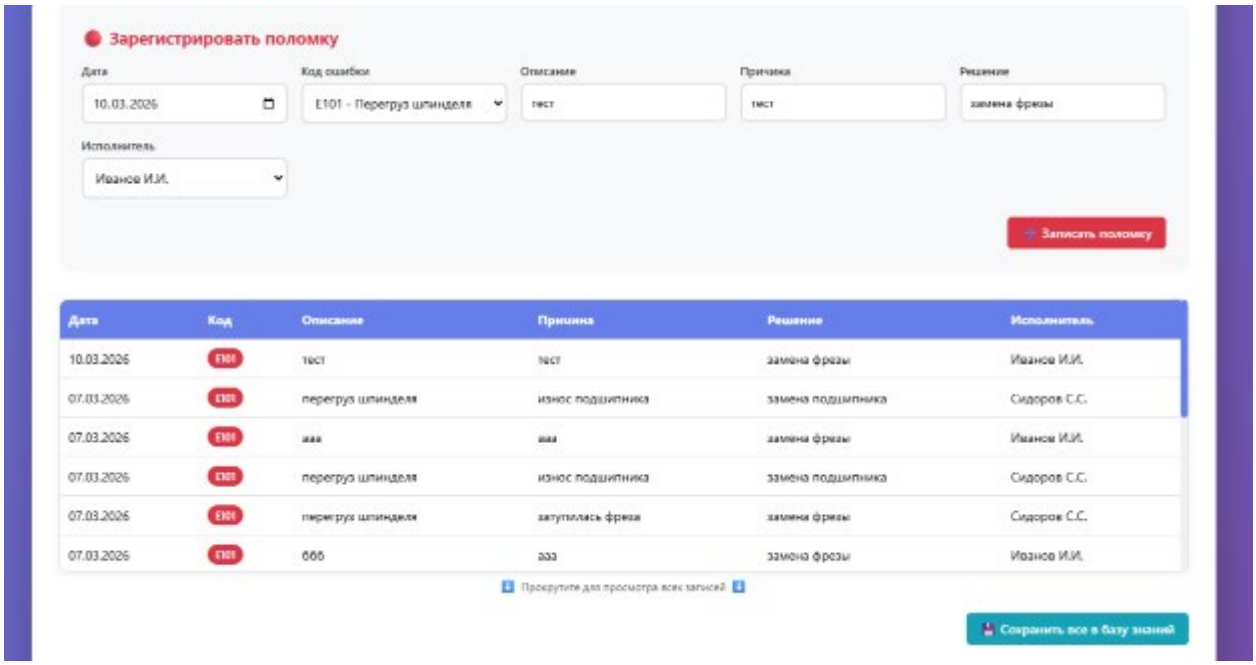


2. Вкладка «Журнал учета»

В форму поломки вносим тестовые данные (описание-тест, причина-тест, решение-замены фрезы).



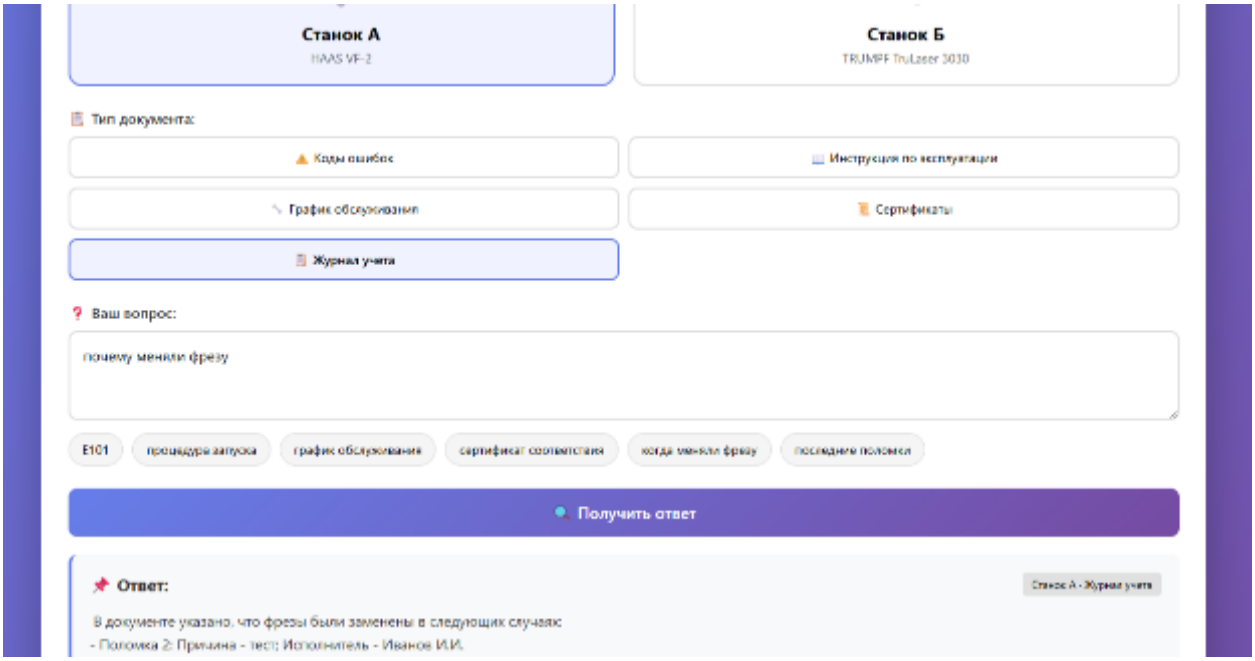
После записи и сохранения данные попадают в таблицу как первая запись (10.03.2026 /тест/тест/замена фрезы/Иванов ИИ).



В Qdrant формируется новая точка Point.

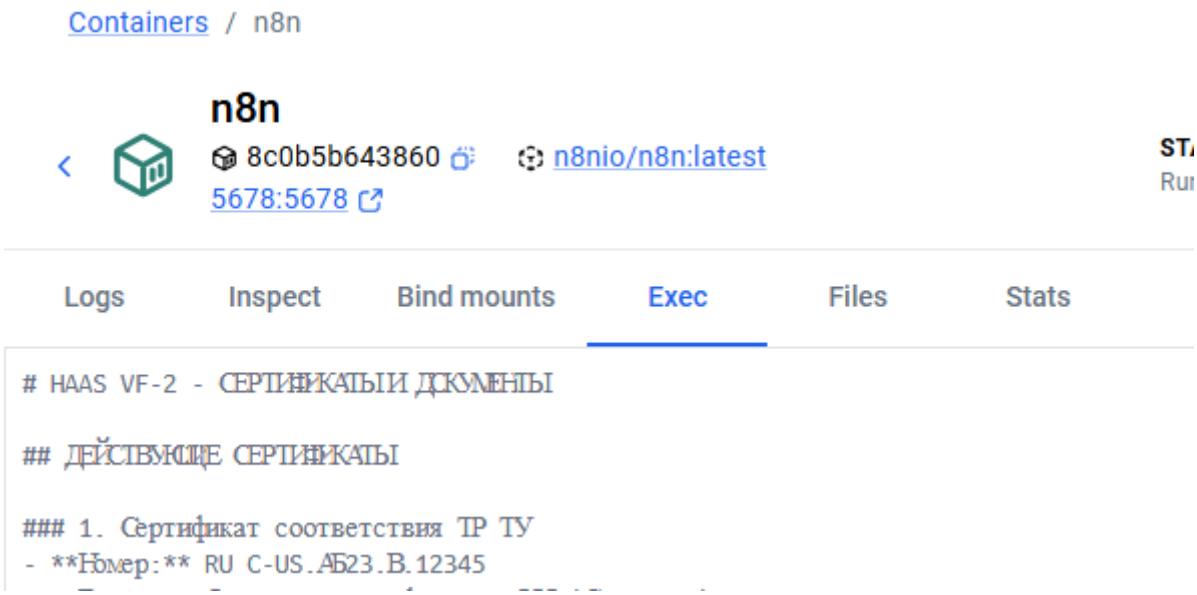


И теперь если в поиске запросить «почему меняли фрезу», то в ответ будут включены и вновь внесенные данные (Причина-тест, Исполнитель-Иванов И.И.).

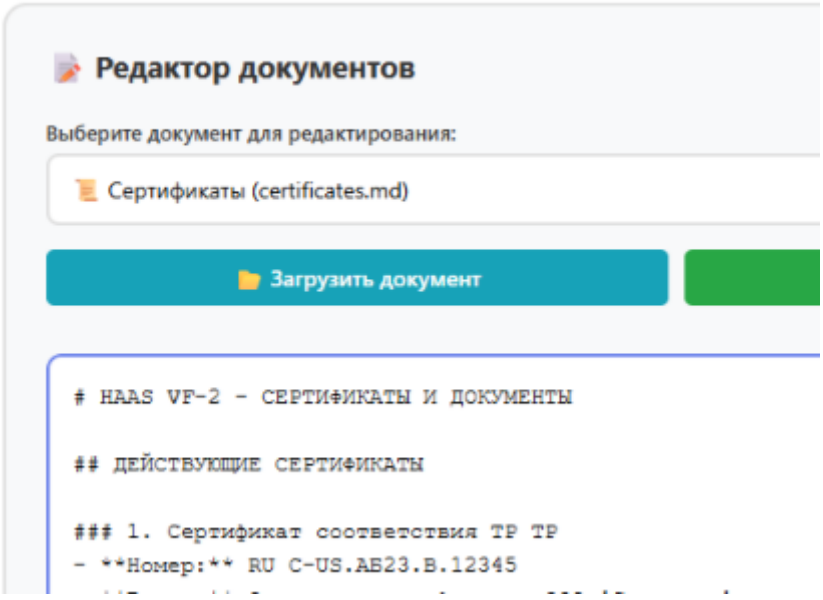


3. Вкладка «Документация»

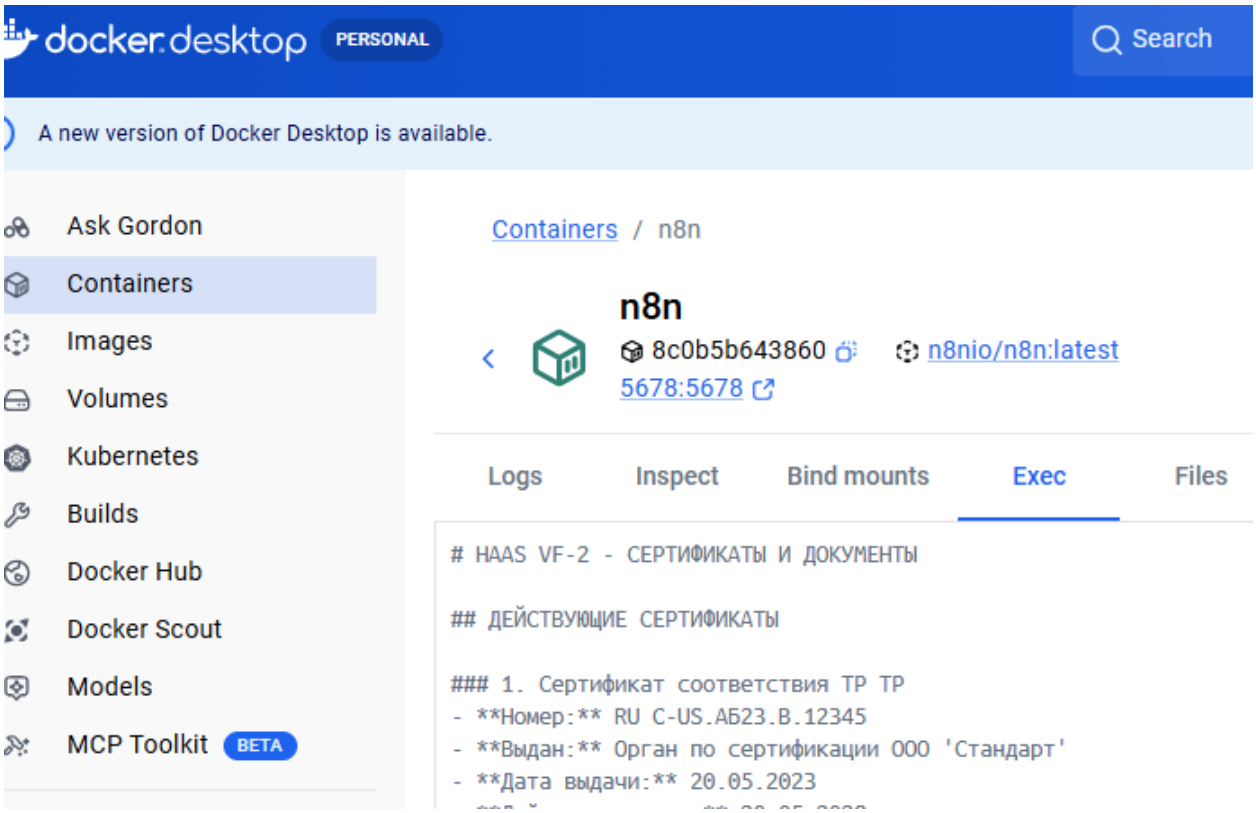
Для просмотра в терминале контейнера n8n файла certificates.md (сертификаты) используем команду less /home/node/.n8n-files/documents/machine_A/certificates.md. И видим, что артикул сертификата **ТР ТУ.**



В интерфейсе загружаем документ и меняем данные на **ТР ТР.** Сохраняем и обновляем базу знаний.



Возвращаемся в Docker и видим, что файл изменился.



Итог: Система готова к масштабированию на другие станки и типы документов.